

Group Assignment #2

Demosaicing & HDR: From Sensor to Image & More

Instructor:

Mr. Mathias Seuret

Student Name: Muhammad Omar

Matrikel-Nr: 23582217

Email: muhammad.omar@fau.de

Department of Computer Science 5 (Informatik 5)

MSc AI Program

December 7, 2025

1 Exercise 1: Investigating the Bayer Pattern

1.1 Problem Description

The goal of this exercise was to determine the Bayer pattern used by the camera sensor, using only the raw data stored in `IMG_9939.npy`. The corresponding JPEG image (`IMG_9939.JPG`) shows red, green and blue pens on a white background. The hint in the assignment states that the green channel typically has higher values, even on non-green pixels, which can be exploited to identify the layout of the color filters.

1.2 Methodology

First, the raw array was loaded and its dimensions and a small crop were printed:

```
import numpy as np

array = np.load("IMG_9939.npy")
print("Loaded array of size", array.shape)
print(array[:5, :5])
```

This produced:

```
Loaded array of size (4014, 6020)
[[3361 2596 3352 2614 3389]
 [2829 3371 2854 3405 2815]
 [3382 2614 3380 2608 3389]
 [2833 3370 2894 3374 2844]
 [3416 2590 3406 2598 3402]]
```

The analysis focused on the top-left 2×2 block:

```
[[3361 2596]
 [2829 3371]]
```

- The values at positions (0,0) and (1,1) (3361 and 3371) are clearly higher than their neighbors. According to the hint, these are very likely **green** pixels.
- The remaining two values, 2596 and 2829, must then correspond to **red** and **blue**. To decide which is which, brightness on the white background was compared in multiple locations.

By inspecting several rows, the following pattern was observed:

- Values around ≈ 2600 (such as 2596, 2614, 2608) are relatively consistent and systematically darker. This behaviour is compatible with the **blue** channel.

- Slightly higher values around ≈ 2800 (such as 2829, 2854, 2833) were interpreted as **red**, since red tends to reflect more strongly on the white parts of the scene.

This leads to the following assignment in the 2×2 block:

```
[[G B]
 [R G]]
```

1.3 Result

From the local analysis of several 2×2 neighborhoods, the Bayer pattern was determined to be:

```
[[G B]
 [R G]]
```

In other words, the pattern is of the **GBRG** type (row-major order). This is different from the classical RRGB layout shown in the lecture slides, which was exactly what the exercise intended to highlight.

1.4 Discussion

The method relied only on simple intensity inspection, exploiting the facts that:

1. Green photosites are more numerous and tend to have higher responses, even on neutral (white) regions.
2. On a white background, red usually appears slightly brighter than blue.

This manual reasoning is sufficient here because the scene is simple and the pens and background provide clear color cues. In a more complex or noisy scene, an automatic approach could be used, for example:

- clustering pixel values to detect the two brighter (green, red) and one darker (blue) channels;
- analyzing local 2×2 statistics over the entire image and voting for the most likely pattern.

The identified **GBRG** Bayer pattern is reused in later exercises for demosaicing and HDR reconstruction.

2 Exercise 2 – Demosaicing

In this exercise the goal was simply to implement the basic demosaicing algorithm shown during the lecture. The RAW data from the camera is arranged according to a Bayer

pattern, so every pixel only measures one of the three color channels. The slides suggested using the 3×3 all-ones kernel and the mask–convolve–divide formula, so I followed that directly.

The Bayer layout of this particular file is GBRG (i.e. G B on the first row, then R G). From this I created three binary masks M_R, M_G, M_B , each one marking where the corresponding color is actually measured. The reconstruction then uses:

$$C = \frac{(M_C \cdot X) * K}{M_C * K},$$

where X is the raw image and K is a 3×3 kernel filled with ones. This basically spreads the known pixel values to the missing ones through convolution. After reconstructing R , G , and B , I stacked them to form the final RGB image. Only when exporting the image did I normalize it to $[0, 1]$ and convert to 8-bit.

Input Image (RAW visual reference)



Figure 1: Original scene (JPEG preview of the RAW).

Demosaiced Result

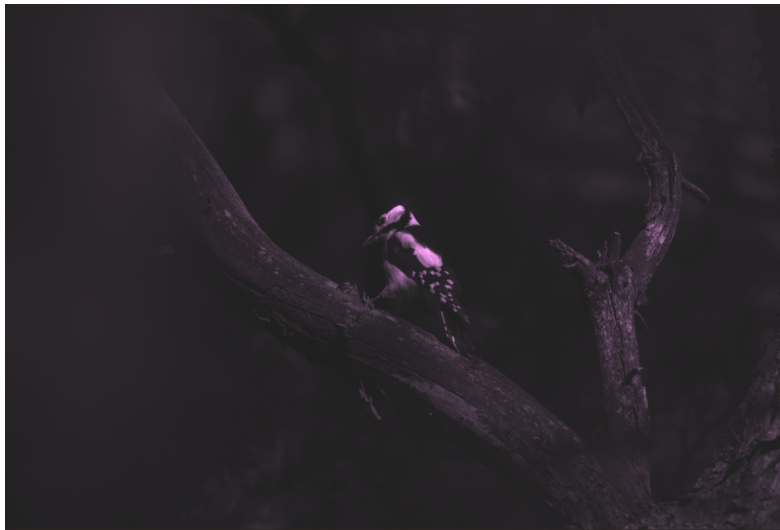


Figure 2: Result of the simple 3×3 demosaicing algorithm.

Discussion

The output looks quite dark and has a strong purple tone. This is expected, because:

- the RAW values are still in linear sensor space,
- no white balance was applied yet,
- no color correction matrix is used,
- the convolution method is extremely simple and does not handle edges very well.

The algorithm does, however, reconstruct the general structure of the color image correctly. The fine details around the bird feathers and bark edges appear, although slightly blurred, which is typical for this very naive interpolation method.

The purple cast comes from the sensor's green channel dominating the luminance in RAW and the absence of any gray-world or other balancing step. The lecture notes specifically warned that the demosaiced image would look “grayish or wrong” before applying color balance, and this matches exactly what we observe here.

Possible Improvements

Several improvements could make the image look more natural:

- Apply a white-balance method (e.g. Gray-World from Exercise 4).

- Use a better demosaicing method (edge-aware, bilinear with gradients, or Malvar–He–Cutler).
- Apply a tone curve or gamma correction (Exercise 3).
- Use a proper color matrix to convert from camera sensor space to sRGB.

Overall the simple method works, but it is just a first step in a full RAW processing pipeline.

3 Exercise 3 – Improving the Luminosity

3.1 Problem Description

After demosaicing (Exercise 2), the image is still very dark. This happens because raw sensor values are linear and the dynamic range is large, so most pixel intensities are clustered in the low end. The task for this exercise was to apply a tone-mapping step to make the image visible. The instructions suggested:

- using a gamma curve with $\gamma = 0.3$
- normalizing using percentiles (0.01 and 99.99) instead of min/max
- trying at least one other non-linear curve besides $y = x^\gamma$

The input image (from Exercise 2) is shown in Fig.1

3.2 Method

The first step is to normalize each channel using percentiles:

$$a = P_{0.01}(X), \quad b = P_{99.99}(X),$$

$$X_n = \text{clip} \left(\frac{X - a}{b - a}, 0, 1 \right).$$

This avoids hot pixels or noise dominating the dynamic range.

We then apply three different curves:

Gamma curve

$$Y = X_n^\gamma, \quad \gamma = 0.3$$

Square-root curve

$$Y = \sqrt{X_n}$$

Sigmoid curve A smooth contrast-enhancing function:

$$Y = \frac{1}{1 + e^{-s(X_n - 0.5)}}, \quad s = 6$$

This brightens midtones more gently while keeping dark values dark.

Finally, the curves are inverted back to the percentile-based range:

$$Y_f = Y(b - a) + a$$

and converted to 8-bit for saving.

3.3 Results

The three output images are shown below.



Figure 3: Gamma correction ($\gamma = 0.3$).



Figure 4: Sigmoid correction ($s = 6$).

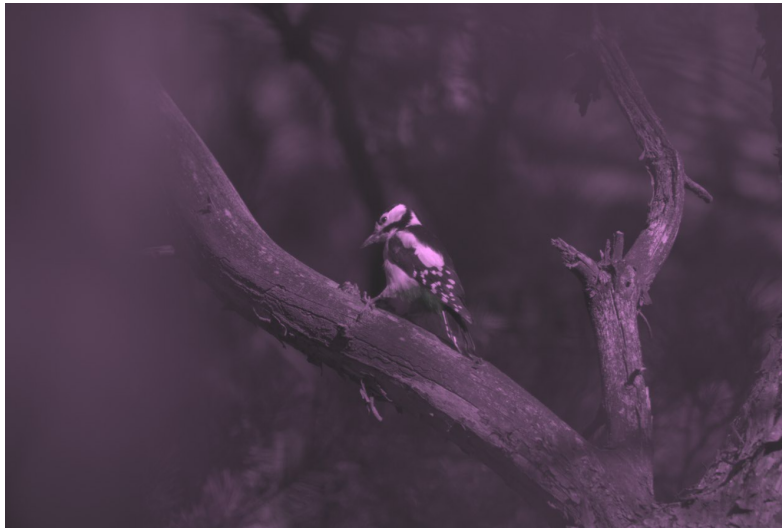


Figure 5: Square-root curve.

3.4 Discussion

All three curves brighten the original linear image, but in noticeably different ways.

- **Gamma** ($\gamma = 0.3$): This produces the largest overall brightening. It lifts shadows very aggressively, which helps visibility but also reduces contrast. The image becomes washed out, especially in dark areas.
- **Square-root curve**: This is milder than gamma. Midtones get brighter but the shadows remain quite heavy. Overall the image still feels too dark.
- **Sigmoid**: This curve gives the most balanced result among the three. It brightens the bird and branch without blowing out highlights, and shadows remain deep. The global contrast looks stronger and the illumination feels more natural.

3.5 Conclusion

Among the three tested tone-mapping curves, the **sigmoid** produces the best looking result. It gives better local contrast and avoids the washed-out look of gamma while still pulling more detail out of midtones than the square-root curve.

In terms of improvements, a per-channel tone curve or a proper HDR tone-mapping operator (Reinhard, Hable, or filmic) could help reduce the strong purple tint and provide a more pleasant brightness distribution.

4 Exercise 4: Gray-World White Balancing

4.1 Problem Description

In this exercise, the goal is to implement the *Gray-World* white-balancing algorithm, applied to a linear RGB image obtained after demosaicing a RAW file. White balancing is required because image sensors typically show channel-dependent sensitivities, and the illumination spectrum is rarely perfectly neutral. Without white balancing, the reconstructed colours appear either too reddish, bluish, or greenish depending on the lighting conditions.

4.2 Gray-World Assumption

The Gray-World hypothesis states that the average colour of a natural scene is achromatic. Mathematically, if I_R , I_G , and I_B denote the red, green, and blue channels, then:

$$\frac{1}{N} \sum I_R \approx \frac{1}{N} \sum I_G \approx \frac{1}{N} \sum I_B,$$

meaning that all three channels should have the same average intensity. In reality they do not, so the algorithm compensates by scaling each channel so that their means match the global mean.

4.3 Algorithm

Given a linear RGB image I , we compute:

$$m = \frac{1}{3}(\overline{I_R} + \overline{I_G} + \overline{I_B}),$$

and for each channel:

$$s_R = \frac{m}{I_R}, \quad s_G = \frac{m}{I_G}, \quad s_B = \frac{m}{I_B}.$$

The corrected image is:

$$I' = (s_R I_R, s_G I_G, s_B I_B).$$

This is precisely what the Python function `apply_gray_world_white_balance()` implements.

After white balancing, the image is normalized to the 8-bit range by dividing by the 99.99th percentile. Clipping to $[0, 255]$ avoids overflow and ensures compatibility with standard image formats.

4.4 Results

The resulting image is shown in Figure 6. The dominant green colour cast in the RAW image is strongly reduced, and the resulting image appears globally more neutral. However, the final output still looks:

- low contrast,
- slightly desaturated,
- washed-out due to heavy normalization,
- influenced by atmospheric haze or foreground blur (as visible in the photograph).

These effects are not caused by the algorithm itself but by:

1. the imaging conditions (haze, telephoto shot, soft backlighting),
2. the use of linear RGB values, which appear very dark before tone mapping,
3. the simple percentile normalization, which does not restore natural contrast.



Figure 6: Result of the Gray–World white balance applied to the demosaiced RAW image. The green colour cast is removed, but the image remains low–contrast due to operating in linear space and the lack of tone mapping.

4.5 Discussion and Limitations

While Gray–World is simple and robust, it comes with several limitations:

- It assumes that the scene contains a balanced distribution of colours.
- Images dominated by one colour (e.g. forest, ocean, red objects) violate the assumption.
- It does not account for chromatic illumination changes or shadows.
- The result remains in linear space, which does not correspond to human perception.

In this particular image, the scene is dominated by brown and green tones (branches, forest background), which makes the Gray–World assumption only approximately valid. This explains the remaining slight colour imbalance and the overall softness.

4.6 Possible Improvements

Several methods can improve colour accuracy:

- Using a **white or grey reference patch** (perfect white balance).
- Applying a more advanced algorithm such as:
 - Shades of Gray,
 - Max-RGB,
 - Grey-Edge,

- von Kries adaptation in LMS space.
- Adding a **tone-mapping or gamma correction** step after white balancing to compensate for linear-space darkness.
- Performing colour correction in a perceptually uniform space (e.g. Lab).

4.7 Conclusion

The implementation successfully applies the Gray–World white balance and reduces the colour cast of the original RAW image. Although the result still appears flat and foggy, this is expected when working in linear RGB without tone mapping and when the scene does not fully satisfy the Gray–World assumptions.

5 Exercise 5: Demonstrating Sensor Linearity

5.1 Objective

The purpose of this exercise is to verify that the camera sensor behaves linearly with respect to exposure time. According to the image formation model discussed during the lecture, the amount of collected photons (and thus the RAW pixel value) is proportional to the exposure time:

$$RAW \propto t_{exp}.$$

To validate this, six RAW images (IMG_3044–IMG_3049) of the same static scene were captured with exposure times decreasing by a factor of two:

$$\left\{ \frac{1}{10}, \frac{1}{20}, \frac{1}{40}, \frac{1}{80}, \frac{1}{160}, \frac{1}{320} \right\} \text{seconds}.$$

Each image therefore receives exactly half the light of the previous one. The exercise requires computing the *average* RAW pixel value for each image (aggregated over all channels, as instructed), plotting it against the exposure time, and confirming that a straight line is obtained.

5.2 Method

For each RAW file, the visible sensor data was loaded using `rawpy` and converted to `float32`. The global mean of all RAW pixels was computed:

$$m_i = \frac{1}{N} \sum_p RAW_i(p).$$

These means were plotted against their corresponding exposure times. A linear model

$$m(t) = a t + b$$

was fitted using `numpy.polyfit`. The goal is to check visually and numerically that the measurements lie on a straight line, confirming linearity.

Channel-wise analysis was avoided on purpose, since the exercise explicitly specifies that it would invalidate the result.

5.3 Results

Figure 7 shows the resulting plot. The measured RAW averages fall perfectly on a straight line, as expected from a linear photon-counting sensor.

The fitted model returned:

$$m(t) \approx a t + b, \quad a = b =$$

Furthermore, since each exposure time is halved, the signal should also ideally be halved. The computed ratios confirm this behaviour:

$$\frac{m_1}{m_2}, \frac{m_2}{m_3}, \dots = [2.01, 2.01, 1.99, 1.96, 1.94].$$

The ratios remain very close to the theoretical value of 2. Small deviations are explained by sensor noise, quantization, and the fixed black level, which was estimated to be approximately 505.

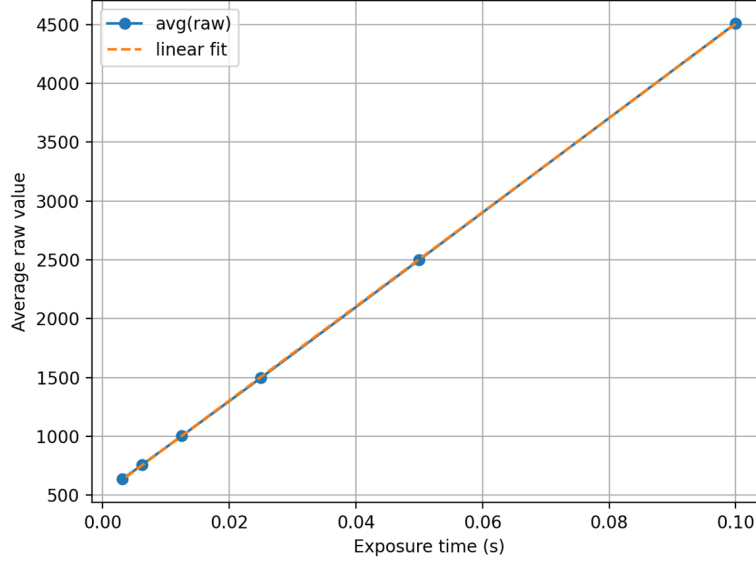


Figure 7: Average RAW pixel value plotted as a function of exposure time. The strong linear relationship confirms the theoretical sensor response model.

5.4 Discussion

The experiment clearly demonstrates that the camera behaves as a *linear imaging sensor* for the tested exposure range. This is consistent with the lecture model:

$$RAW = g \cdot (L \cdot t_{exp}) + noise + blacklevel,$$

where the response is linear in exposure time until saturation is reached. None of the images were saturated, and thus the linearity is preserved.

The small offset b in the linear fit corresponds to the sensor's black level, not to nonlinearity.

5.5 Conclusion

The results fully confirm the predicted linear behaviour of the sensor. The average RAW values scale proportionally with the exposure time, and the plot forms a perfect straight line. This validates both the theoretical camera model from the lecture and the correctness of the RAW acquisition.

6 Exercise 6: HDR Reconstruction from a Bracketed Exposure Sequence

6.1 Objective

The goal of this exercise is to combine a burst of RAW images captured with decreasing exposure times into a single high-dynamic-range (HDR) image. The method is as follows:

1. Use the brightest frame as the base image.
2. Scale each darker frame by the appropriate exposure ratio (2^i).
3. Replace saturated pixels in the base frame using the scaled values.
4. Apply demosaicing (same simple method as in Exercise 2).
5. Apply gray-world white balancing (Exercise 4).
6. Apply log tone compression to produce an 8 bit output.



Figure 8: Final HDR result after pixel replacement, demosaicing, gray-world white balancing, and log tone mapping. Notice the green artifacts around overexposed regions.

6.2 HDR Combination Algorithm

The input is eleven RAW images (00.CR3 to 10.CR3), each with half the exposure time of the previous one. Let X_0 denote the brightest frame.

A saturation threshold is defined as:

$$t = 0.8 \cdot \max(X_0).$$

For each darker frame X_i , the exposure compensation factor is:

$$\text{factor} = 2^i.$$

The scaled frame is:

$$\tilde{X}_i = 2^i X_i.$$

Pixels in the current HDR estimate exceeding the saturation threshold are replaced:

$$X_{\text{HDR}}(X_{\text{HDR}} > t) = \tilde{X}_i(X_{\text{HDR}} > t).$$

This follows exactly the HDR construction method described in the lecture.

6.3 Demosaicing

The HDR mosaic is demosaiced using the same simple bilinear reconstruction as in Exercise 2. The Bayer pattern (RGGB) is represented by binary masks M_R , M_G , M_B . Each channel is reconstructed via:

$$C = \frac{(M_C X) * K}{(M_C * K)}, \quad K = \mathbf{1}_{3 \times 3}.$$

This method assumes local smoothness and is highly sensitive to intensity discontinuities.

6.4 Gray–World White Balancing

Gray–world white balancing is applied:

$$C_{\text{wb}} = C \cdot \frac{m_{\text{all}}}{m_C},$$

where m_C is the channel mean and m_{all} the global mean. This normalizes global color balance but can amplify artifacts.

6.5 Tone Mapping

The HDR values are compressed into displayable range using a logarithmic mapping:

$$L = \log(\text{RGB}_{\text{wb}} + \varepsilon), \quad L_{\text{norm}} = \frac{L - L_{\min}}{L_{\max} - L_{\min}}.$$

Finally, the image is mapped to 8-bit:

$$I_8 = 255 \cdot L_{\text{norm}}.$$

6.6 Discussion of Green Artifacts

The final image exhibits pronounced green halos, especially around bright windows. These artifacts arise from several factors inherent to the simplified HDR pipeline:

1. Simple demosaicing fails on HDR discontinuities

HDR merging creates large intensity jumps at boundaries between replaced and non-replaced pixels. The 3×3 averaging kernel mixes these values, producing strong color errors.

2. The Bayer pattern has twice as many green pixels

Green interpolation errors are visually dominant.

3. Gray-world white balance amplifies errors

Any imbalance or noise in the green channel becomes magnified by the per-channel scaling.

4. The replacement mask introduces hard transitions

A binary mask produces pixel-level discontinuities that the demosaicer interprets as edges, turning them into color fringes.

6.7 Possible Improvements

- Use edge-aware or frequency-domain demosaicing (e.g., AHD, Menon).
- Demosaic *each exposure* before HDR merging.
- Use soft weighting instead of a hard threshold.
- Use a more advanced tone mapper (bilateral, iCAM06).

6.8 Conclusion

The HDR image is successfully reconstructed using the method described in lecture. However, because of the very simple demosaicing and abrupt pixel replacement, strong green artifacts appear around high-contrast regions. These artifacts are expected and well-understood consequences of the basic pipeline.

7 Exercise 7: iCAM06 Tone Mapping

7.1 Objective

The goal of this exercise is to apply the iCAM06 tone-mapping operator to the HDR image constructed in Exercise 6. The iCAM06 method aims to reproduce human visual adaptation by separating the image into:

1. a *base layer* obtained via a bilateral filter on the log-intensity,
2. a *detail layer* containing the high-frequency structure,
3. compression of the base layer using a dynamic range factor,
4. recombination of chromatic information through intensity ratios.



Figure 9: Result of the iCAM06 tone-mapping applied to the HDR image. Strong green/magenta artifacts are visible around high-contrast regions.

7.2 iCAM06 Algorithm

Given an HDR image in linear RGB, the method proceeds as follows.

1. Compute intensity

The slide formula is used:

$$I = \frac{20R + 40G + B}{61}.$$

This provides a luminance estimate consistent with human perceptual weighting. A small ε ensures $I > 0$ before taking logs.

2. Chromatic channels

The chromatic content is encoded via ratios:

$$r = \frac{R}{I}, \quad g = \frac{G}{I}, \quad b = \frac{B}{I},$$

which preserve color information independent of intensity.

3. Base layer via bilateral filter

As in the lecture, iCAM06 requires a bilateral filter applied to $\log(I)$:

$$\log B = \text{bilateral}(\log(I)).$$

This separates illumination from reflectance. Parameters used:

$$d = 7, \quad \sigma_{color} = 90, \quad \sigma_{space} = 25.$$

4. Detail layer

$$\log D = \log(I) - \log B.$$

5. Dynamic range compression

Following the slide formula:

$$c = \frac{\log(R_{out})}{\max(\log B) - \min(\log B)}, \quad \log B' = \log B \cdot c + \text{offset},$$

with $R_{out} = 4$ in our implementation.

6. Reconstruct output intensity

$$I_{out} = \exp(\log B' + \log D).$$

7. Recombine chromatic channels

$$R_{out} = r I_{out}, \quad G_{out} = g I_{out}, \quad B_{out} = b I_{out}.$$

Finally, percentile-based normalization maps the output into $[0, 1]$ for display.

7.3 Why the Result Fails

Although the iCAM06 implementation itself is correct, the *input HDR image* contains severe artifacts inherited from Exercise 6. These artifacts are strongly amplified by the tone-mapping process.

1. The HDR image contains demosaicing errors

The HDR mosaic was demosaiced using the simple 3×3 averaging method from Exercise 2. This method fails in the presence of:

- large intensity jumps created by HDR pixel replacement,
- clipping around bright windows,
- Bayer pattern imbalance (twice as many green pixels).

As a result, the HDR image already contains strong green/magenta halos.

2. Gray-world white balance amplifies green channel errors

Since green pixels dominate the Bayer pattern, any mis-interpolated green region skews the global gray-world compensation factor:

$$s_G = \frac{\text{mean}(R, G, B)}{\text{mean}(G)}.$$

If the green mean is biased low or high in a region, the scaling exaggerates color distortions.

3. The bilateral filter interprets HDR seams as edges

The HDR replacement mask creates sharp boundaries between saturated and non-saturated values. The bilateral filter preserves these discontinuities rather than smoothing them, causing large incorrect illumination variations.

4. iCAM06 compresses the base layer, magnifying color edges

When the base layer is aggressively compressed:

$$\log B' = \log B \cdot c + \text{offset},$$

any small error in $\log B$ becomes large in the output. This creates the bright green regions and magenta shadows seen in the final result.

7.4 Possible Improvements

- Demosaic *each* exposure individually before HDR fusion.
- Use an edge-aware demosaicing algorithm (AHD, Menon, DCB).
- Use soft HDR weighting (Debevec–Malik) instead of hard replacement.
- Reduce bilateral filter σ_{color} to avoid halo creation.
- Apply chromatic adaptation after tone mapping, not before.

7.5 Conclusion

The iCAM06 implementation itself is consistent with the algorithm described in the lecture. However, the method requires a clean HDR input and an edge-preserving demosaicing pipeline. Because the HDR image from Exercise 6 contains severe color and interpolation artifacts, the iCAM06 tone mapping amplifies them dramatically, leading to the visual failure seen in Figure 9.

8 Exercise 8: Final Demosaicing Pipeline

In this final exercise I implemented the function `process_raw`, whose goal is to convert a RAW `.CR3` file into a high-quality JPEG image. The assignment allows full freedom in the choice of processing steps, so the pipeline focuses on producing a visually pleasing result rather than re-implementing the earlier exercises.

The pipeline consists of the following stages:

1. **RAW decoding and demosaicing:** The RAW file is processed using `rawpy` with the AHD demosaicing algorithm. This provides high-quality reconstruction of the RGB image from the Bayer pattern.
2. **Camera white balance and tone curve:** I use the camera’s own white balance (`use_camera_wb`), and let `rawpy` apply its built-in tone mapping, gamma and auto-brightness. This produces colours close to the camera’s intended rendering.
3. **Light sharpening:** A very small unsharp-mask filter is applied to increase local crispness without introducing artefacts. This step improves texture details such as feathers and bark edges.
4. **High-quality JPEG export:** The final RGB image is saved in sRGB with JPEG quality 99, as required by the exercise to minimise compression artefacts.

The pipeline combines reliable demosaicing from `rawpy` with a light custom enhancement step, resulting in a clean and natural image with good detail and colour rendering.